GATE Prep Desk

Architecture, flows and memory design of a fully local AI tutor

A GATE Computer Science tutor that runs entirely on one Windows laptop: a browser app, a small PowerShell server, Ollama for the language models, stable-diffusion.cpp for pictures, and a real C compiler for checking programs.

Prepared 19 October 2026. Ollama 0.40.6 on an NVIDIA RTX 5050 Laptop GPU (8 GB). All memory and speed figures were measured on this laptop from Ollama's own log, except where marked "calculated".

Contents

1. Overview 3
2. System architecture 4
   2.1 Components 4
   2.2 How the parts talk 4
3. Start-up and shutdown 5
4. Front-end modules 5
5. Server API 6
6. Data and storage 6
7. How the app uses Ollama 7
   7.1 Models and their roles 7
   7.2 The request 7
   7.3 Why one fixed context size 7
   7.4 The running Ollama configuration 7
8. How a model is loaded into memory 8
9. VRAM and RAM budget 9
10. KV cache and context 10
   10.1 What the KV cache is 10
   10.2 KV cache per model 10
   10.3 Filling the context window 11
11. Main flows 12
   11.1 Learn a topic 12
   11.2 Photo doubt and answer checking 12
   11.3 Practice and the mistake loop 13
   11.4 Tutor chat 13
   11.5 Previous-year questions 13
   11.6 Night shift 13

12. Visual system 13
13. Performance and limits 14
14. Tuning recommendations 14
Appendix A. Hardware and software 15
Appendix B. Glossary 15

1. Overview

GATE Prep Desk teaches GATE CS topics, explains photographed doubts, builds practice tests and mocks, tracks mistakes with spaced review, and studies uploaded books and previous-year papers. The whole application is a set of plain HTML, CSS and JavaScript files (about 14,000 lines) that run in a browser window. A PowerShell server stores the data and runs helper programs; Ollama runs the language models on the GPU. Nothing else needs installing, and no data leaves the laptop unless the optional web lookup is switched on, in which case only the topic or question is sent.

Design principles

- **Local first.** Every model, file and check runs on this laptop; the app works offline.
- **Verify before trusting.** A small model writes the teaching, but answers are checked by code wherever possible: exact calculators, a real C compiler, SQLite, automata and data-structure simulators, independent re-solves, a larger second model, and official GATE keys when uploaded.
- **One fast model for most work.** qwen3.5:4b generates about 62 tokens per second fully on the GPU; the 12-billion-parameter model is used only for second opinions.
- **Exact visuals.** Trees, graphs, automata, Gantt charts and flowcharts are drawn by code from data, so every node and number is right. AI pictures are used only as memory aids.
- **No build step.** Files are served as written; the libraries (pdf.js, KaTeX, Mermaid, Tesseract.js, sql.js and others) are bundled for offline use.

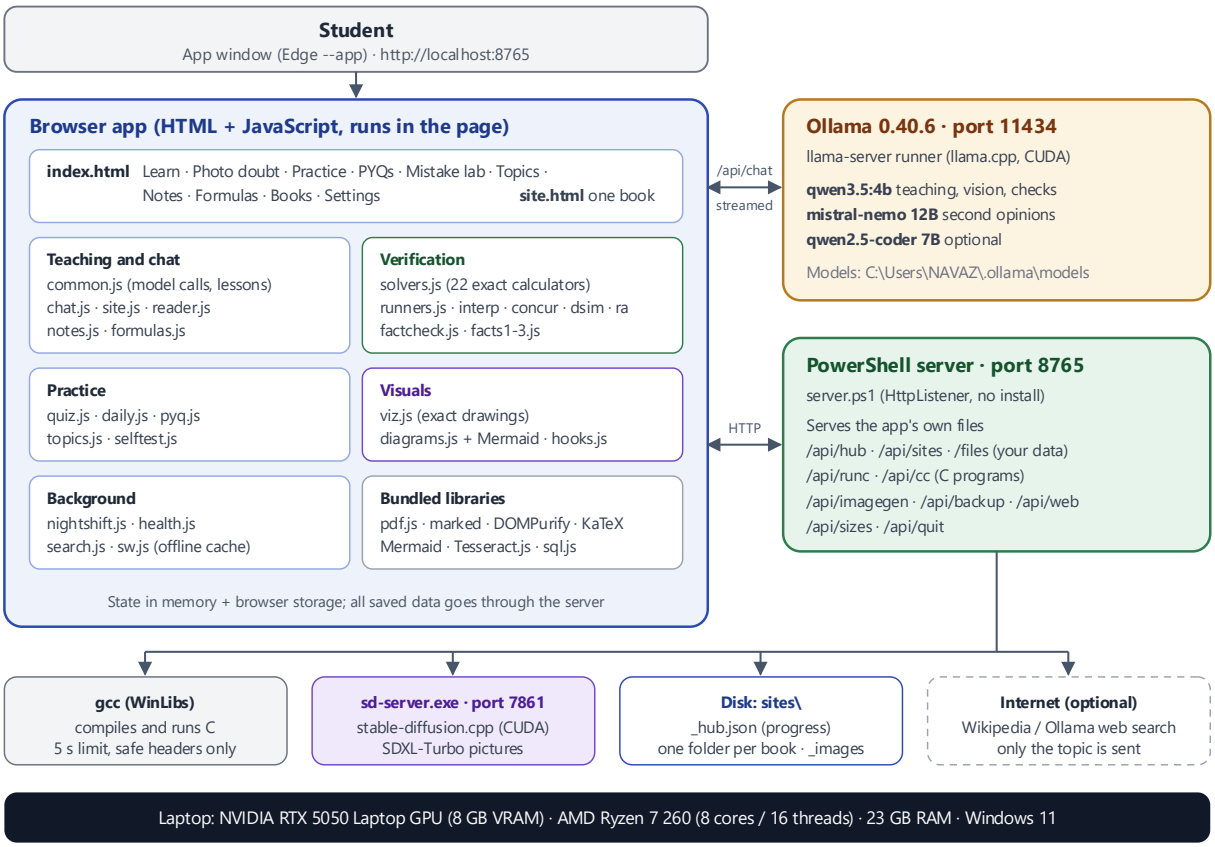


Figure 1. System architecture: the browser app talks to Ollama directly for model calls and to the PowerShell server for data, C programs and pictures.

2. System architecture

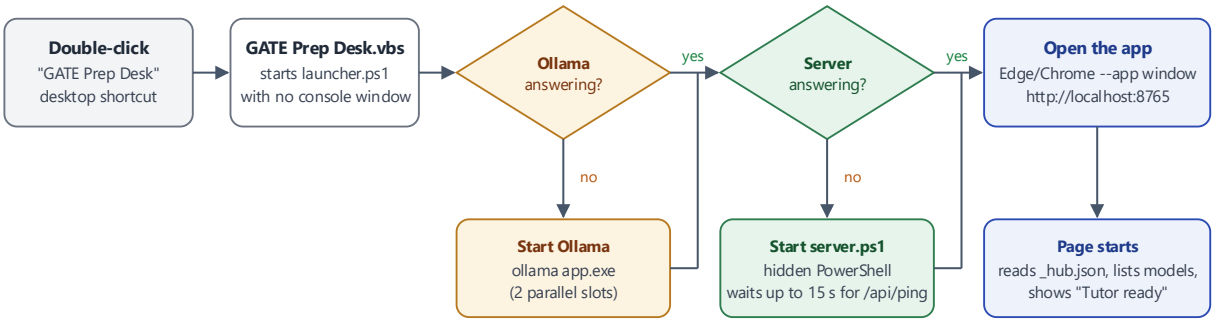
2.1 Components

Component	Runs as	Role
Browser app	A page in Edge or Chrome, opened as an app window (--app)	All features and screens; builds prompts; runs the exact checks; calls Ollama directly
PowerShell server (server.ps1)	Hidden powershell.exe with an HttpListener on port 8765	Serves the app files; reads and writes your data; compiles and runs C; starts the picture engine; backups; web-lookup proxy
Ollama 0.40.6	Tray app plus a llama-server.exe runner per loaded model, port 11434	Loads models into GPU and RAM; streams tokens; enforces JSON schemas
sd-server.exe	stable-diffusion.cpp, CUDA build (Vulkan fallback), port 7861, started on first use	Draws memory-hook pictures with SDXL-Turbo
gcc (WinLibs, UCRT)	Started by the server for each request	Compiles and runs C programs with a 5-second limit and a header allow-list
Data on disk	sites\ folder next to the app	Progress file, one folder per uploaded book, pictures, web cache
Internet (optional)	Wikipedia, or Ollama web search with a key	Extra details with sources; only the topic is sent

2.2 How the parts talk

- **Page to Ollama:** POST http://localhost:11434/api/chat with stream set to true. Tokens arrive as one JSON object per line and are drawn as they come. Ollama already allows the localhost origin, so no proxy is needed.
- **Page to server:** fetch() calls to /api/... for everything that is stored or executed; uploaded files are read from /files and /data.
- **Server to helpers:** gcc and sd-server.exe run as child processes; the server talks to sd-server through its /sdapi/v1/txt2img endpoint and saves the PNG under sites_images.
- **Service worker (sw.js):** app files are fetched network-first (always fresh, with a cached copy as fallback), bundled libraries cache-first, and /api is never cached. It also makes the app installable.

3. Start-up and shutdown



Models are not loaded at start-up: Ollama loads a model on the first request that names it, then keeps it for 5 minutes after its last use.
Stopping: Settings → Shut down calls /api/quit, which also stops the image generator (sd-server.exe). Ollama keeps running in the tray.

Figure 2. Start-up: the desktop shortcut starts whatever is not already running, then opens the app window.

1. The "GATE Prep Desk" shortcut runs GATE Prep Desk.vbs, which starts launcher.ps1 without a console window.
2. If Ollama does not answer on port 11434, the launcher starts the Ollama tray app (with OLLAMA_NUM_PARALLEL=2 when it is the one starting it).
3. If the server does not answer /api/ping, it starts server.ps1 hidden and waits up to 15 seconds.
4. It opens http://localhost:8765 in an Edge or Chrome app window. The page loads the progress file, lists the installed models and shows "Tutor ready".
5. No model is loaded yet: Ollama loads one on the first request that names it.

Settings, Shut down calls /api/quit, which stops the server and the picture engine. Ollama keeps running in the tray and unloads its model after 5 idle minutes.

4. Front-end modules

Area	Files	What they do
Core	common.js, theme.js, palette.js	Model calls (ollamaChat), Markdown and KaTeX rendering, the step-by-step lesson viewer, confidence labels, settings, background jobs and notifications
Pages	hub.js (index.html), site.js (site.html)	Learn, Photo doubt, Practice, PYQs, Mistake lab, Topics, Notes, Formulas, Books, Settings; one book's study space
Teaching	chat.js, reader.js, notes.js, formulas.js, topics.js	Tutor chat, reading PDFs and photos (pdf.js, Tesseract, vision model), notes, formula book, topics and mastery checks
Exact checks	solvers.js, runners.js, interp.js, concur.js, dsim.js, ra.js	22 calculators (CPU and disk scheduling, paging, cache, TLB, subnetting, K-maps, CRC, IEEE 754, grammars, minimal DFA, graphs, functional dependencies, probability and more); runners for C, SQL, Boolean expressions, regular expressions and grammars; a pseudo-code interpreter with static/dynamic scoping and every parameter-passing mode; concurrency interleavings; data-structure simulators; relational algebra
Fact-checking	factcheck.js, facts.js, facts2.js, facts3.js	Verified GATE reference facts; a reviewer plus per-claim truth checks (local and web); recomputation of every written calculation; diagram repair

Area	Files	What they do
Practice	quiz.js, daily.js, pyq.js, selftest.js	Question writing and answer-key checks, the exam screen, grading, daily mix, the PYQ bank and full-paper mode, the accuracy self-test
Visuals	viz.js, diagrams.js, hooks.js	Exact SVG drawings, Mermaid flowcharts, memory-hook pictures
Background	nightshift.js, health.js, search.js, sw.js	Idle-time jobs, data health checks, search across everything, the offline cache

5. Server API

Endpoint	Purpose
GET / and /<file>	The app's own files, sent with Cache-Control: no-store
GET /vendor/..., /icons/...	Bundled libraries and icons, cached for a year
GET / PUT /api/hub	The progress file, sites_hub.json
/api/sites, /data, /files	Uploaded books: metadata, the text read from each page, the original files
POST /api/runc, GET /api/cc	Compile and run a C program (header allow-list, forbidden calls such as system() and fopen(), 5-second limit); compiler status
POST /api/imagegen, /api/imagegen/status, /api/imagegen/stop	Draw a picture; picture-engine status; unload the engine
GET /api/image/<name>.png, POST /api/image/delete	Serve and delete saved pictures
/api/backup, /api/restore, /api/autobackup	Full .zip backup and restore; weekly automatic backup to a folder (newest 4 kept)
/api/topics-file, /api/notes-file, /api/formulas-file	Keep the Markdown copies in the app folder up to date
/api/web	Web lookup (Wikipedia, or Ollama web search with a key)
/api/sizes, /api/ping, /api/quit	Disk use per book; liveness check; shut down (also stops sd-server)

6. Data and storage

- **sites_hub.json** (about 250 KB today): lessons and topics, the covered list, test attempts, mistakes and their review schedule, flashcards, PYQs, formulas, notes, chats, settings, confidence calibration and fact-check results. Every change goes through the server.
- **sites\<book id>** for each upload: the original PDF or images, meta.json, and data.json with the text read from every page (Markdown), its lessons and its tests.
- **sites_images** holds the memory-hook pictures; sites_webcache.json caches web lookups.
- **Markdown copies:** My GATE topics.md, My GATE notes.md and My GATE formulas.md in the app folder, readable without the app.
- **Backups:** one .zip with everything (manual, or weekly to a chosen folder such as a USB drive). Restoring first keeps a copy of the current data.

- **Models:** C:\Users\NAVAZ\.ollama\models (Ollama GGUF files) and tools\sd\models (SDXL-Turbo checkpoint and VAE).

7. How the app uses Ollama

7.1 Models and their roles

Model	On disk	Architecture	Used for
qwen3.5:4b	3.10 GiB, Q4_K_M	4.2 B parameters; 32 layers (8 full attention + 24 Gated DeltaNet); vision encoder; tools; thinking	Default for teaching, chat, reading images, checks and every JSON task
mistral-nemo 12B (2407)	6.96 GiB, Q4_K_M	12.2 B parameters; 40 attention layers; 8 KV heads	Second opinions when confidence is low
qwen2.5-coder 7B	4.36 GiB, Q4_K_M	7.6 B parameters; 28 attention layers; 4 KV heads	Optional; selectable in Settings

Settings can assign separate models to teaching, reading images, checking answers and second opinions. Q4_K_M stores most weights in about 4.5 to 5 bits each, so a 4-billion-parameter model fits in about 3 GB.

7.2 The request

- Endpoint: POST /api/chat with stream: true. The page reads the response line by line and shows tokens as they arrive.
- think: false. qwen3.5 can reason in a hidden <think> block first; the app turns this off for speed and strips any such block.
- options.num_ctx = 16,384 for every call; options.num_predict caps the answer (4,000 tokens by default, 3,000 in the tutor chat); temperature 0 for checks, 0.3 by default, up to 0.7 for creative drafts.
- format: a JSON schema for structured work (questions, flashcards, fact checks, picture scores). Ollama constrains decoding so the reply always parses as that shape.
- images: base64 images attached to a message go to the vision model.
- keep_alive is not sent, so Ollama's default of 5 minutes applies.

7.3 Why one fixed context size

Ollama sizes the KV cache for num_ctx when it loads a model. A request with a different num_ctx forces a full reload, which takes 10 seconds or more. The app therefore uses 16,384 tokens everywhere: enough for about 20,000 characters of notes or book pages plus a long, detailed lesson.

7.4 The running Ollama configuration

Setting	Value	Effect
Version	0.40.6	llama.cpp-based runner (llama-server.exe)
OLLAMA_NUM_PARALLEL	1 (2 when the app's launcher starts Ollama)	Each parallel slot needs its own KV cache

Setting	Value	Effect
Flash attention	auto (enabled for these models)	Attention computed in tiles: smaller compute buffers, faster prompt reading
OLLAMA_KV_CACHE_TYPE	not set (f16)	2 bytes per stored value; q8_0 would halve the cache
OLLAMA_KEEP_ALIVE	5 minutes	An idle model is unloaded after 5 minutes
OLLAMA_MAX_LOADED_MODELS	auto	Several models can stay loaded if they fit together
GPU	CUDA on the RTX 5050 (compute capability 12.0); Vulkan also enabled	Free VRAM seen at load: 6.9 GiB, 457 MiB kept spare
Default context length	262,144	Overridden by the app's num_ctx of 16,384

8. How a model is loaded into memory

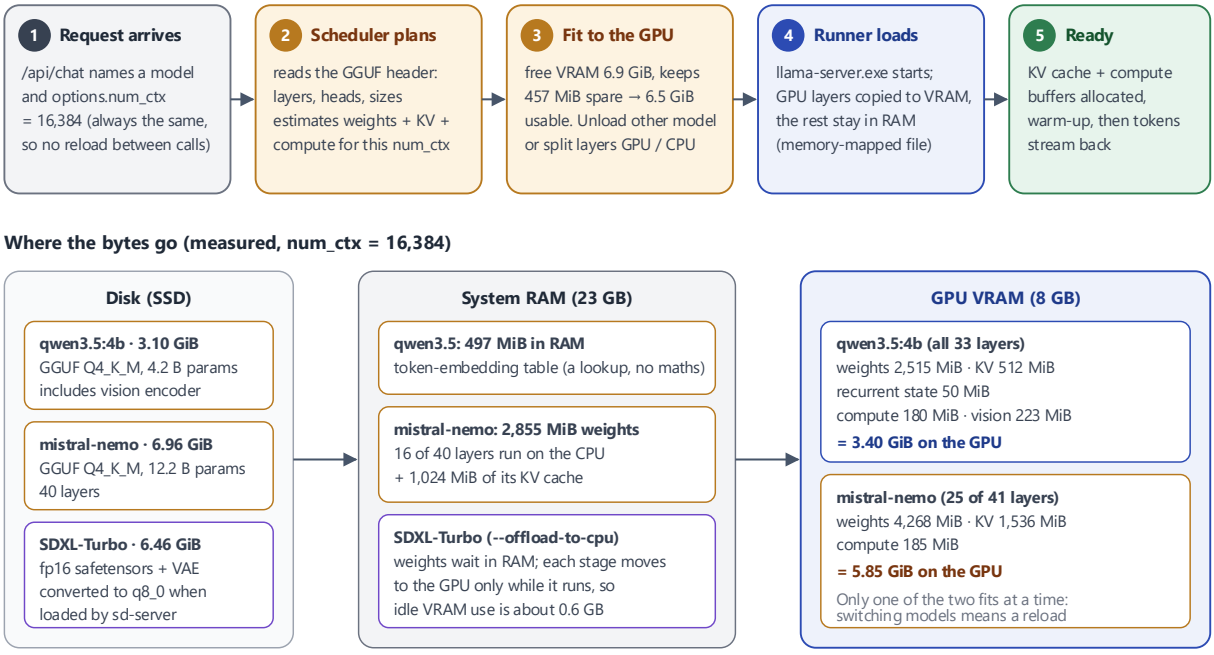


Figure 3. From a request to a loaded model, and where each model's bytes end up.

6. A request names a model and a context size. If that model is already loaded with the same settings, it is used straight away.
7. Ollama's scheduler reads the GGUF file header (architecture, number of layers, heads and tensor sizes) and estimates the memory for the weights, the KV cache for num_ctx times the parallel slots, and the compute buffers.
8. It checks free GPU memory: 6.9 GiB free, of which it keeps 457 MiB as a margin, leaving about 6.5 GiB usable. Idle models are unloaded first if the new one does not fit beside them.
9. If the whole model fits, every layer goes to the GPU. If not, it splits by layer: as many layers as fit are placed in VRAM and the rest run on the CPU from RAM.
10. A llama-server.exe runner starts. The GGUF file is memory-mapped from disk; GPU layers are copied into VRAM. The token-embedding table stays in host memory because it is only looked up, never multiplied.

11. The KV cache, the recurrent state (for hybrid models) and the compute buffers are allocated, a warm-up pass runs, and tokens start streaming.
12. The model stays loaded for 5 minutes after its last request. Asking for a model that cannot fit alongside it triggers an unload and a load.

Measured on this laptop: qwen3.5:4b places all 33 of its layer groups on the GPU. mistral-nemo places 25 of 41 on the GPU and runs 16 on the CPU.

9. VRAM and RAM budget

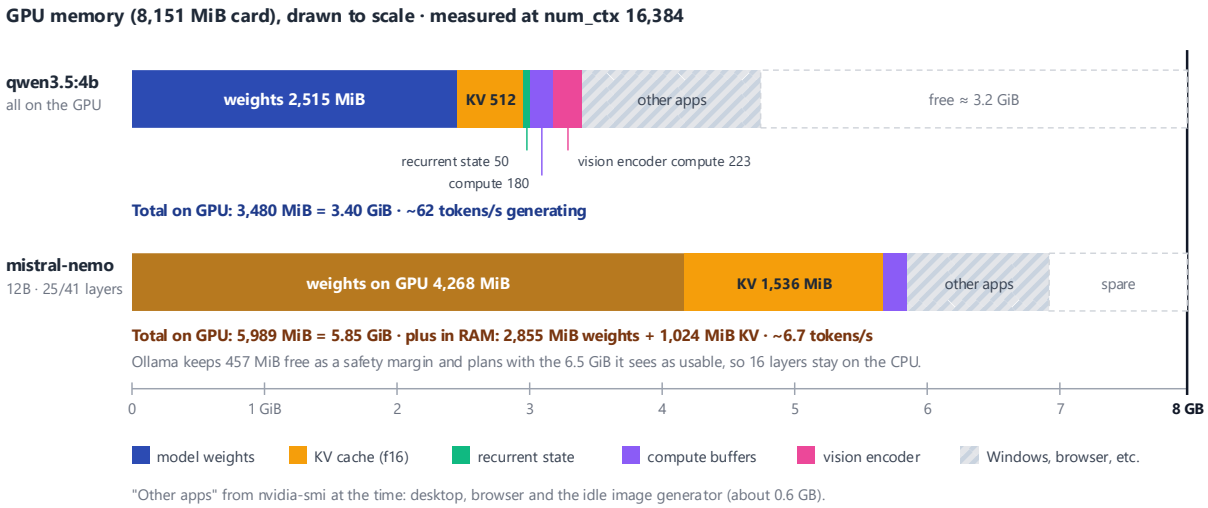


Figure 4. GPU memory drawn to scale for each model at num_ctx 16,384.

Buffer (MiB)	qwen3.5:4b	mistral-nemo 12B
Weights on the GPU	2,515	4,268
Weights in RAM	497 (token embeddings)	2,855 (16 layers on the CPU)
KV cache (f16)	512, all on the GPU	2,560 (1,536 GPU + 1,024 RAM)
Recurrent state	50	none
Compute buffers	180 GPU + 72 host	185 GPU + 56 host
Vision encoder compute	223 (worst case estimated at 892)	none
Total on the GPU	3,480 = 3.40 GiB	5,989 = 5.85 GiB
Generation speed	about 62 tokens/s	about 6.7 tokens/s
Prompt reading speed	about 2,200 tokens/s	about 480 tokens/s

qwen3.5:4b fits completely with about 3 GiB to spare, so the desktop, the browser and the idle picture engine share the GPU without slowing it down. mistral-nemo does not fit: 16 of its layers and 1 GiB of its KV cache live in RAM and run on the CPU, which makes it about nine times slower. Only one of the two can be loaded at a time, so each second opinion swaps the models, and the Ollama log shows these reloads repeating.

10. KV cache and context

10.1 What the KV cache is

A transformer reads a prompt once, and for every token it has seen, each attention layer keeps that token's key (K) and value (V) vectors. A new token then attends to these stored vectors instead of recomputing the whole past. This store is the KV cache. Its size is: attention layers x 2 (K and V) x KV heads x head size x bytes per value x tokens. Ollama allocates it up front for the whole num_ctx when the model loads, so a longer context costs memory even before it is used.

Grouped-query attention (GQA) shrinks it: in qwen3.5 sixteen query heads share four K/V heads, and in mistral-nemo thirty-two share eight.

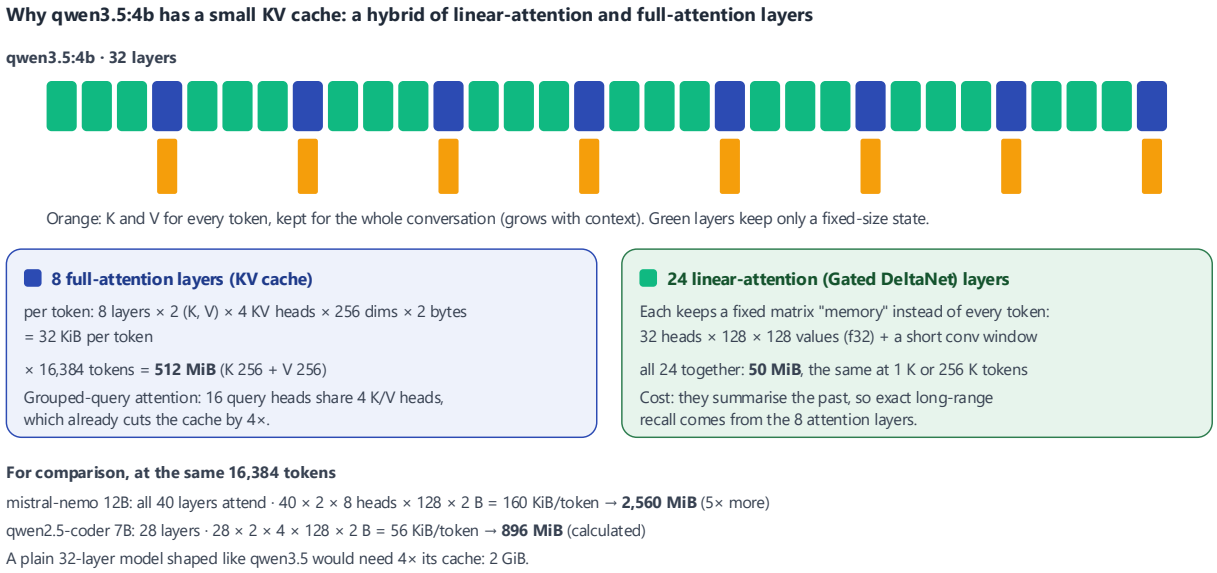


Figure 5. qwen3.5:4b keeps a KV cache in only 8 of its 32 layers; the other 24 keep a small fixed-size state.

10.2 KV cache per model

Model	Attention layers	KV heads x head size	Per token	At 16,384 tokens
qwen3.5:4b	8 of 32	4 x 256	32 KiB	512 MiB (measured)
qwen2.5-coder 7B	28	4 x 128	56 KiB	896 MiB (calculated)
mistral-nemo 12B	40	8 x 128	160 KiB	2,560 MiB (measured)

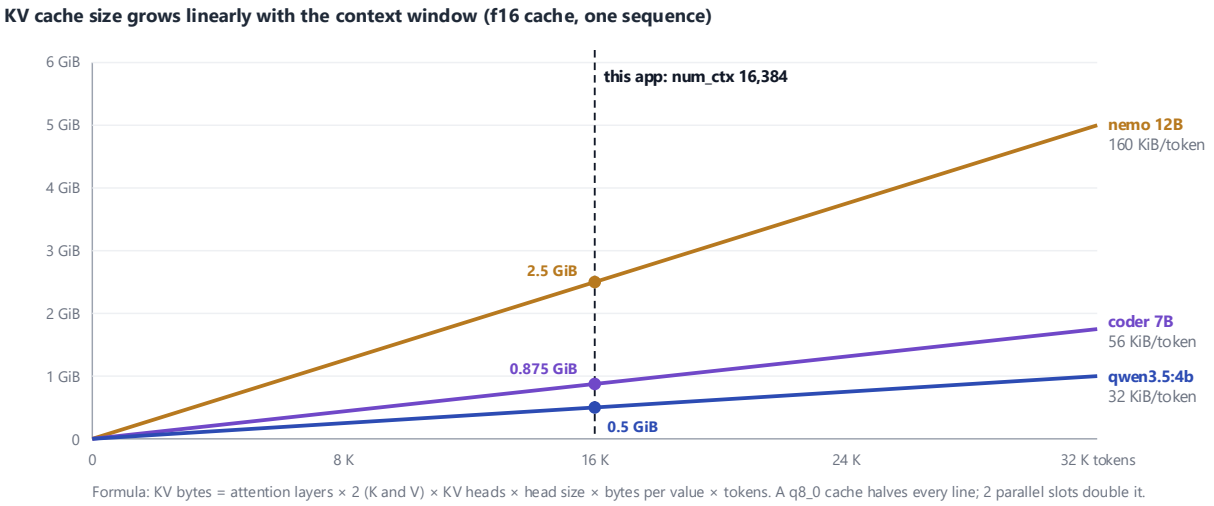


Figure 6. KV cache size against context length for the three installed models.

Context (tokens)	4,096	16,384 (app)	32,768	65,536	131,072	262,144 (max)
qwen3.5:4b KV cache	128 MiB	512 MiB	1 GiB	2 GiB	4 GiB	8 GiB: does not fit

The 24 Gated DeltaNet layers of qwen3.5 replace the growing cache with a fixed matrix memory per layer (32 heads x 128 x 128 values in 32-bit floats, plus a short convolution window): 50 MiB in total whatever the context length. This is why the model can offer a 262,144-token window, and why 16,384 tokens cost it only 512 MiB.

10.3 Filling the context window

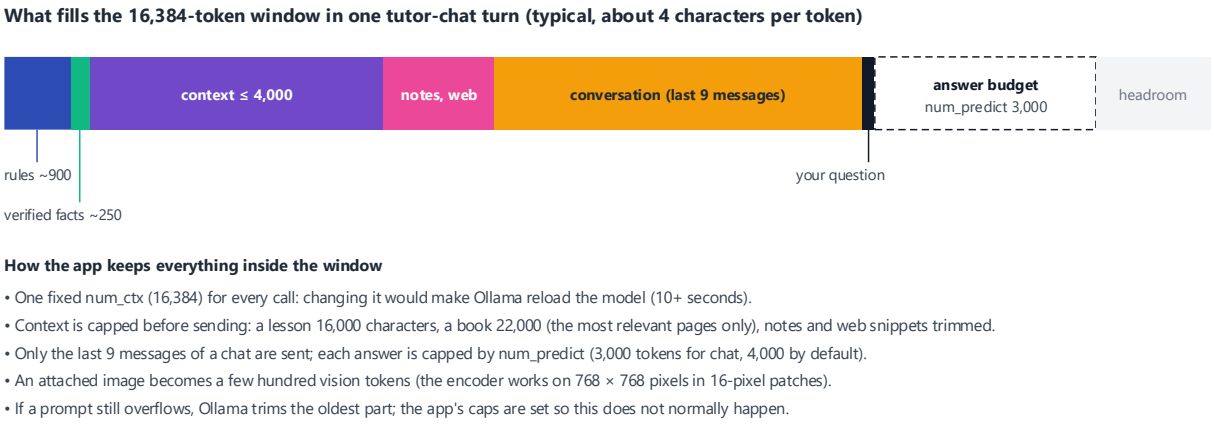


Figure 7. A typical tutor-chat turn inside the 16,384-token window.

Every prompt is assembled from the task rules, a few verified GATE facts, the chosen context (a lesson is capped at 16,000 characters, a book at 22,000 characters of its most relevant pages), the student's notes and web snippets when relevant, the last 9 chat messages, and the question, leaving room for the answer. An attached image is encoded by the vision tower (24 layers working on 768 x 768 pixels in 16-pixel patches) into a few hundred tokens.

11. Main flows

11.1 Learn a topic

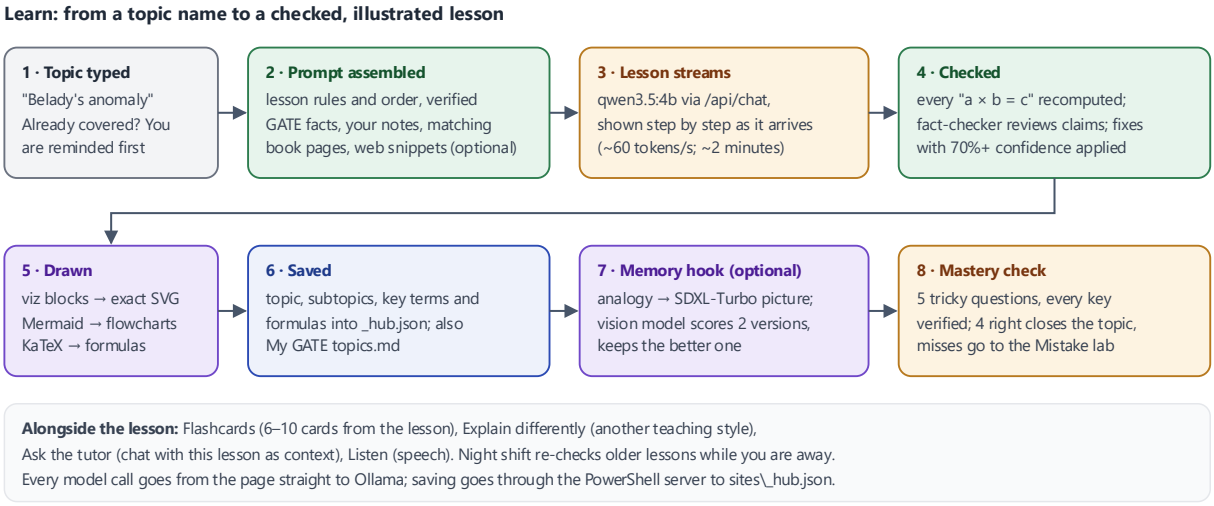


Figure 8. Learn: a lesson is written, checked, drawn and saved, followed by a mastery check.

The lesson prompt fixes the order of teaching (the big picture, prerequisites checked against your topics, then a deep step-by-step explanation with diagrams, formulas, traps and a quick check) and asks for exact drawings as viz data and flowcharts as Mermaid. After streaming, every written calculation is recomputed, the fact-checker reviews the claims (with web evidence when online) and applies only corrections it is at least 70% sure of; the rest are flagged for the student.

11.2 Photo doubt and answer checking

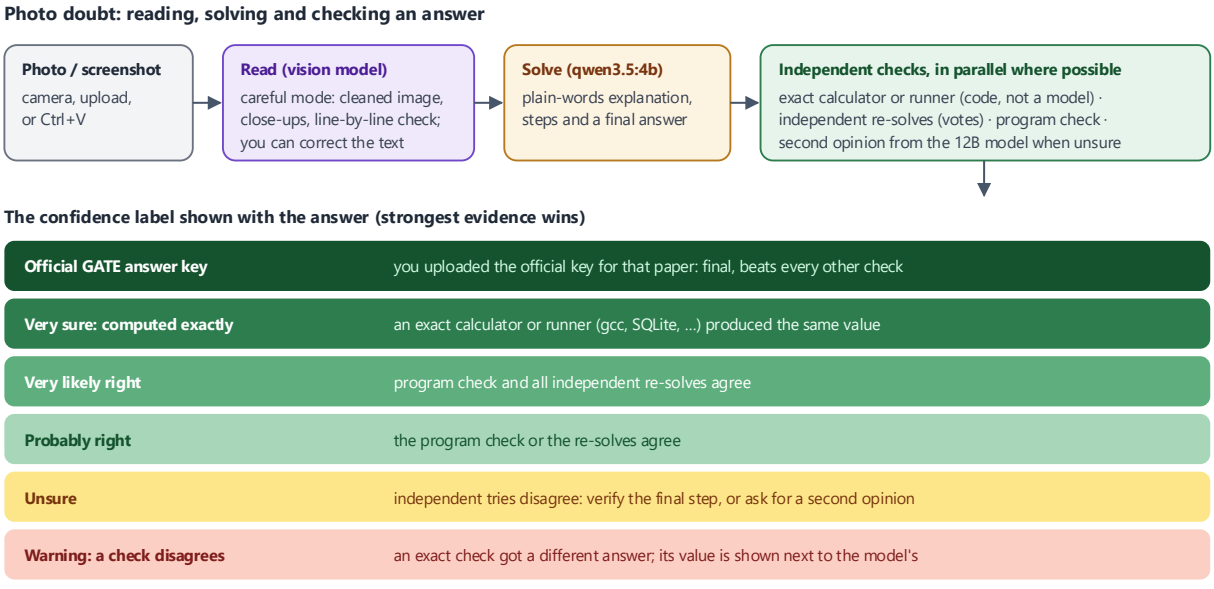


Figure 9. Photo doubt and the confidence ladder shown beside every answer.

The careful reading mode cleans the image, reads close-ups and checks every line against the photo, and the student can correct the text before it is solved. The answer is then checked in parallel by an exact

calculator or runner when one applies, by independent re-solves, by a program check and, when still unsure, by the 12B model. The label reflects the strongest evidence, and Settings, Reliability shows how often each label turned out right per subject.

11.3 Practice and the mistake loop

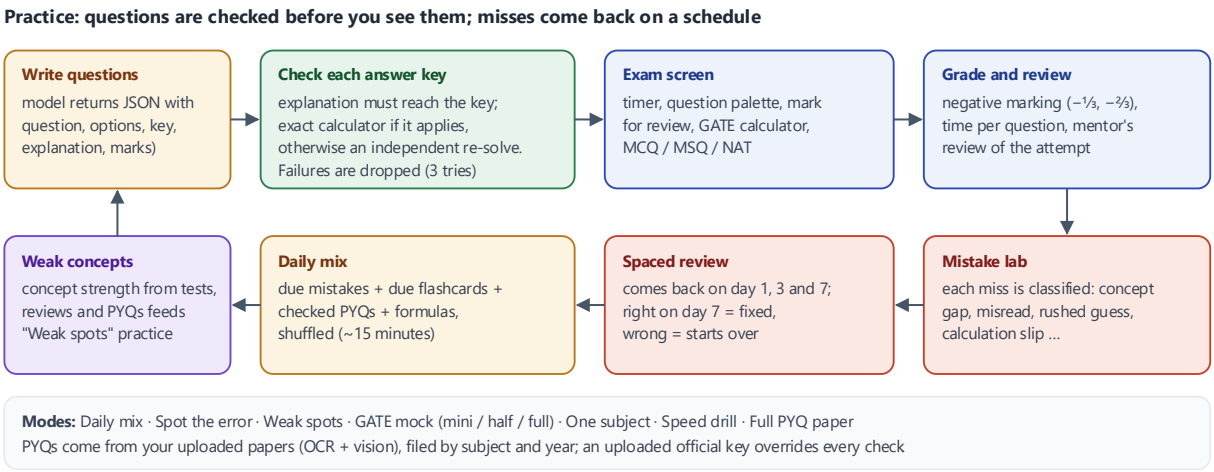


Figure 10. Practice: questions are verified before use; misses return on day 1, 3 and 7.

11.4 Tutor chat

- A side panel on every page. The + button attaches images (sent to the vision model) and picks the context: this lesson, page, chapter or whole book, your notes, your topics, or none. "What's sent" shows the exact text.
- Code, SQL, Boolean expressions and grammars in a question are run for real first (gcc, SQLite and the runners), and the answer must agree with the real output; it is asked again if not, then flagged.
- Answers stream with formulas, diagrams and code blocks (each with a Copy button), and can be copied, retried or fact-checked. Three follow-up questions are suggested.

11.5 Previous-year questions

Uploaded papers are read page by page in the background (the PDF text layer when present, otherwise Tesseract OCR plus the vision model), split into questions and filed by subject and year. Each PYQ is taught as a masterclass lesson. An uploaded official answer key overrides every other check and is used to score full-paper mode with GATE marking.

11.6 Night shift

After 3 minutes without input, and with no other job running, the app works through slow tasks: checking PYQ answers, fact-checking older lessons, mini lessons and flashcards, and writing PYQ masterclasses ahead. It stops as soon as the student returns and saves as it goes.

12. Visual system

- viz.js draws trees, graphs, DFA and NFA, Gantt charts, arrays with pointers, stacks, queues, memory layouts and pipeline diagrams from small JSON descriptions, with step-by-step frames for algorithms.

Text is measured in the real font so labels never overlap, and every drawing can be zoomed, shown full screen or saved as SVG or PNG.

- Mermaid draws flowcharts, state and sequence diagrams; the app colours shapes by meaning (start and end, decisions, input and output) and repairs older broken diagram code when it is shown.
- KaTeX typesets formulas offline.

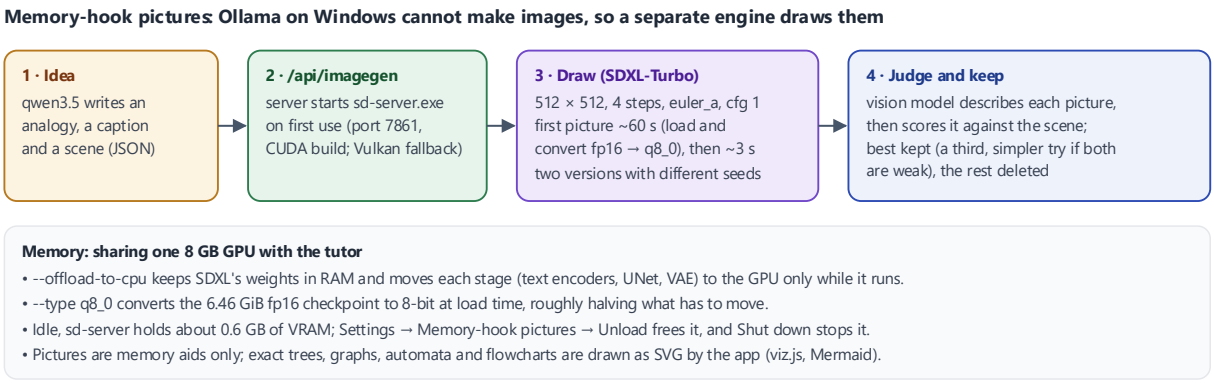


Figure 11. Memory-hook pictures come from a separate engine that shares the GPU with the tutor.

13. Performance and limits

Task	Typical time on this laptop
A full lesson (qwen3.5:4b)	about 2 minutes, plus about 20 seconds of fact-checking
Photo doubt explanation	about 40 seconds
Five checked practice questions	2 to 3 minutes (each key check about 40 seconds)
Tutor chat answer	10 to 25 seconds
Second opinion (mistral-nemo)	a model swap plus about 6.7 tokens per second
Memory-hook picture	about 60 seconds for the first, then about 3 seconds each

- The built-in self-test (47 GATE questions with known answers) scored 62% with the model alone and 85% with all the checks.
- A 4B model can slip on long multi-step arithmetic and long worked examples; the exact checks, the fact-checker and the confidence labels exist for this reason.
- OCR can garble a scanned question; an uploaded official key or re-reading the page fixes it.

14. Tuning recommendations

- **Quantise the KV cache.** Setting OLLAMA_KV_CACHE_TYPE=q8_0 (flash attention is already on) halves every KV cache: about 1.25 GiB less for mistral-nemo, so more of its layers fit on the GPU and second opinions get faster; 256 MiB less for qwen3.5. The quality cost is usually very small.
- **Keep num_ctx at 16,384.** Doubling it adds 512 MiB for qwen3.5 and 2.5 GiB for mistral-nemo, which would push even more of the larger model onto the CPU.
- **Use two parallel slots only if needed.** OLLAMA_NUM_PARALLEL=2 lets two jobs run at once but doubles the KV cache.
- **Make second opinions cheaper.** Each second opinion unloads qwen3.5 and loads mistral-nemo, which then runs partly on the CPU. No second model fits beside qwen3.5 in 6.5 GiB, but one that fits

on the GPU by itself runs many times faster: qwen2.5-coder 7B needs about 4.4 GiB of weights plus 0.9 GiB of cache. Settings, Compare models measures whether it is accurate enough.

- **Longer keep-alive.** OLLAMA_KEEP_ALIVE=30m avoids a reload after short breaks.
- **Unload the picture engine** when not drawing (Settings, Memory-hook pictures, Unload) to free about 0.6 GB of VRAM.

Appendix A. Hardware and software

Item	Value
GPU	NVIDIA GeForce RTX 5050 Laptop GPU, 8,151 MiB, driver 592.82, compute capability 12.0
CPU	AMD Ryzen 7 260 with Radeon 780M, 8 cores / 16 threads
RAM	23.1 GB
OS	Windows 11
Ollama	0.40.6 (llama.cpp runner, CUDA and Vulkan back ends)
Picture engine	stable-diffusion.cpp sd-server.exe (CUDA build), SDXL-Turbo 1.0 fp16 with the fp16-fix VAE
C compiler	gcc from WinLibs (POSIX threads, UCRT)
Browser	Microsoft Edge or Google Chrome

Appendix B. Glossary

Term	Meaning
GGUF	The single-file model format used by llama.cpp and Ollama: weights plus metadata such as layer counts and the tokenizer
Q4_K_M	A 4-bit quantisation scheme (with some tensors at higher precision) that shrinks a model to about a quarter of its 16-bit size
num_ctx	The context window in tokens for which Ollama allocates the KV cache
num_predict	The maximum number of tokens generated for one answer
KV cache	The stored keys and values of every earlier token, per attention layer
GQA	Grouped-query attention: several query heads share one key/value head, shrinking the KV cache
Gated DeltaNet	A linear-attention layer that keeps a fixed-size memory matrix instead of one entry per token
Offloading	Placing some layers (or the whole model) on the GPU and running the rest on the CPU
Flash attention	An attention algorithm that works in tiles, avoiding the full attention matrix in memory
VRAM	The GPU's own memory: 8 GB on this laptop